

MODULE 12 · DOMAIN 2 - DATA STORE MANAGEMENT

Cataloguing and lineage

The Glue Data Catalog, crawlers, partitions, schema evolution,
SageMaker Catalog and data lineage.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 12 of 27

A catalog is what turns **files in a bucket** into **tables you can query**.

Two different kinds of catalog

TECHNICAL CATALOG

AWS Glue Data Catalog

- Table names, columns, types, locations
- Partitions and their physical paths
- Read by query engines to plan queries
- Machine-facing. Nobody browses it for insight

BUSINESS CATALOG

Amazon SageMaker Catalog

- What a dataset means and who owns it
- Glossary terms, descriptions, classifications
- Searched by people looking for data
- Human-facing. Built on top of the technical one

THE TIEBREAKER

The exam guide names both, in separate skills. Confusing them is the most common error in this area.

What the Glue Data Catalog holds

- ◆ **Databases**, which are namespaces rather than anything physical.
- ◆ **Tables**, with columns, data types, and the location of the data.
- ◆ **Partitions**, each mapping a set of key values to a physical path.
- ◆ **Connections**, holding how to reach a JDBC or other external source.
- ◆ **Column statistics**, which query engines use to build better plans.
- ◆ **It holds no data.** Deleting a table deletes metadata, not files.

ON THE EXAM

It is Hive metastore compatible, which is why EMR, Spark and Hive can all use it directly.

Reaching sources through the catalog

- ◆ **A Glue connection** stores how to reach a JDBC source, and lives in the catalog.
- ◆ **Credentials come from AWS Secrets Manager**, referenced rather than stored.
- ◆ **One connection, many consumers.** Crawlers and jobs both use it.
- ◆ **A crawler over a JDBC connection** catalogs a database's schema without moving data.
- ◆ **Network reachability still applies.** The VPC and security group rules from module two.

ON THE EXAM

Cataloguing a relational source lets you see its schema without loading any of it.

How tables get into the catalog

- ◆ A **crawler** scans the data, infers a schema, and creates or updates the table.
- ◆ A **Glue job** registers its own output when `enableUpdateCatalog` is set.
- ◆ **DDL statements** in Athena or Spark create tables explicitly.
- ◆ **Iceberg and other table formats** register themselves as they are written.
- ◆ **AWS DMS** can populate the catalog when writing to Amazon S3.

ON THE EXAM

Four of these avoid running a crawler at all. That is usually the lower-overhead answer.

What a crawler actually does

- ◆ **Samples files** under a path and infers column names and types.
- ◆ **Detects partition structure** from Hive-style folder names.
- ◆ **Groups similar files** into one table, or creates several if they differ.
- ◆ **Billed per DPU-hour with a ten minute minimum**, unlike jobs.
- ◆ **Classifiers** tell it how to read formats it would otherwise guess at.

ON THE EXAM

A crawler infers. Inference is a guess, and on messy data it guesses differently as the data changes.

Crawler settings that decide behaviour

- ◆ **Schema change policy.** Update the table, or ignore the change and log it.
- ◆ **Deleted object behaviour.** Delete the table, mark it deprecated, or ignore.
- ◆ **Crawl new folders only** for incremental crawls on a growing dataset.
- ◆ **Recrawling everything** on a large lake is slow and expensive.
- ◆ **Table level** controls how deep in the path a table boundary sits.

ON THE EXAM

The schema change policy decides whether an upstream change quietly rewrites your table definition.

Where else a catalog can live

OPTION	CHARACTER
AWS Glue Data Catalog	The default. Hive metastore compatible, regional
An external Hive metastore	A self-managed metastore, often inherited from a Hadoop estate
Catalog federation	Glue presents an external metastore as if it were native
Iceberg REST catalog	The open standard, used by SageMaker Lakehouse

ON THE EXAM

Federation is the migration answer. It lets engines query an existing Hive metastore without moving the metadata first.

A Glue job writes daily Parquet output to Amazon S3 under a Hive-style date partition. A crawler is scheduled every fifteen minutes to register new partitions so that analysts can query the current day with Amazon Athena.

Finance has queried the Glue bill. The crawler accounts for a large share of it, and analysts still complain that new data takes up to fifteen minutes to become visible.

What should the data engineer change?

- ☐ A Reduce the crawler schedule to every five minutes so data becomes visible sooner
- ☐ B Enable `enableUpdateCatalog` on the Glue job so it registers its own partitions, and remove the crawler
- ☐ C Configure the crawler to crawl new folders only, and keep the fifteen minute schedule
- ☐ D Switch the table to partition projection in Athena and delete the registered partitions

◆ PAUSE AND DECIDE

What should the data engineer change?

- A

 Crawl every five minutes
Triples the number of ten-minute billing minimums, to improve a latency the job could remove entirely.
- C

 Incremental crawls
Genuinely reduces crawl time and cost, and keeps both the component and the fifteen minute delay.
- B

`enableUpdateCatalog`, remove the crawler
The partition is registered as it is written, so data is visible immediately and the crawler cost disappears.
- D

 Partition projection
A real answer for a predictable date pattern, and it is Athena-only. Redshift and EMR reading the same table lose partition awareness.

THE SIGNAL

Cost and latency both improve by removing a component rather than tuning it. That shape recurs across this exam.

A partition the catalog does not know about is **invisible**, however real the files are.

Five ways to keep partitions current

APPROACH	CHARACTER
Crawler on a schedule	Discovers anything. Slow, billed per run, delayed
enableUpdateCatalog in the job	Registered at write time. No extra component
ALTER TABLE ADD PARTITION	Explicit and precise. Someone has to run it
MSCK REPAIR TABLE	Scans the location and adds everything missing. Slow on large tables
Partition projection	Athena calculates paths from a pattern. No partitions stored at all

ON THE EXAM

Partition projection removes the metadata lookup entirely, and it only works in Athena.

When partition metadata becomes the bottleneck

- ◆ **Tens of thousands of partitions** make lookup slow before any data is read.
- ◆ **Glue partition indexes** speed up lookup on large tables.
- ◆ **Partition projection** avoids the lookup entirely, in Athena.
- ◆ **Over-partitioning** is the underlying cause, as module eight described.
- ◆ **Coarser partitions** are usually the real fix. Monthly instead of hourly.

ON THE EXAM

A query slow before it reads any data, on a heavily partitioned table, is a metadata problem not a data problem.

An hourly ingestion process writes files to Amazon S3 under year, month, day and hour partitions. The table now holds around ninety thousand partitions covering ten years. Analysts query it exclusively by date range, never by hour.

Athena queries take twelve to twenty seconds before scanning begins, regardless of how narrow the date filter is. Once scanning starts, queries complete quickly.

What is the most effective change?

- ☐ A Add a Glue partition index on the partition keys
- ☐ B Repartition the data by year, month and day only, removing the hour partition
- ☐ C Increase the Athena workgroup's query timeout
- ☐ D Run MSCK REPAIR TABLE to refresh the partition metadata

◆ PAUSE AND DECIDE

What is the most effective change?

A Add a partition index

Speeds up lookup against ninety thousand partitions. It optimises around a partition count that should not exist.

C Longer timeout

Lets the slow query finish rather than making it fast. It treats the symptom as acceptable.

B Drop the hour partition

Ninety thousand partitions become under four thousand. Nobody queries by hour, so nothing is lost and lookup shrinks twenty four fold.

D MSCK REPAIR TABLE

Adds missing partitions. Nothing is missing here, and running it across ninety thousand partitions is itself slow.

THE SIGNAL

"Never by hour" is the planted detail. The partition scheme is finer than any query needs.

Schemas change because **the world changes**. The question is what breaks when they do.

What each change does to a pipeline

CHANGE	TYPICAL CONSEQUENCE
A column is added	Usually harmless. Old files return null for it
A column is removed	Queries naming it fail. Others carry on
A column is renamed	Looks like a drop plus an add. History appears to lose the column
A type changes	The dangerous one. Reads may fail, or coerce silently
Nesting changes	Flattening logic breaks, often without an obvious error

ON THE EXAM

Type changes are the ones that produce wrong numbers rather than failures. Those are worse.

How formats handle change

FORMAT	SCHEMA EVOLUTION
CSV	None. Position based. An inserted column shifts everything
JSON	Self describing, so additions are painless. No enforcement at all
Avro	Designed for it. Readers and writers negotiate compatible schemas
Parquet	Schema in the footer. Additions fine, renames are not
Iceberg	Column IDs. Add, drop, rename and reorder are all safe

ON THE EXAM

This is why module nine said Iceberg solves schema evolution. Column IDs are the mechanism.

The Glue Schema Registry

- ◆ **Stores schemas centrally**, with versions, for streams and pipelines.
- ◆ **Producers validate against the registered schema** before sending.
- ◆ **Compatibility rules** reject a change that would break existing consumers.
- ◆ **Works with Kinesis, Amazon MSK, Flink and Lambda.**
- ◆ It governs the stream. It does not fix data already written.

ON THE EXAM

This stops a breaking change at the producer, rather than discovering it in a broken consumer at three in the morning.

When the data changes without the schema

- ◆ **Volume grows**, so a job that finished in an hour now takes four.
- ◆ **Cardinality shifts**, so a key that was evenly spread becomes skewed.
- ◆ **Value distribution changes**, so statistics go stale and plans get worse.
- ◆ **A new dominant value appears**, creating a hot partition where there was none.
- ◆ **Monitor the shape of the data**, not only whether the job succeeded.

ON THE EXAM

The skill says "changes to the characteristics of data", not just the schema. This slide is the rest of it.

Making the catalog itself governable

- ◆ **Lake Formation permissions** are granted on catalog tables and columns.
- ◆ **Tag-based access control** attaches policy to tags rather than to each table.
- ◆ **The catalog is the join point** between storage, engines and permissions.
- ◆ **Reading around the catalog bypasses all of it**, as module two's sixth scenario showed.
- ◆ Register the S3 location with Lake Formation so permissions actually bind.

ON THE EXAM

Every fine-grained permission in Domain 4 is granted on a catalog object. That is why cataloguing comes first.

A Glue job reads JSON events from a partner and writes Parquet to a curated table. Last week the partner began sending a field that had always been an integer as a quoted string for about two percent of records, to represent an unknown value.

The job still reports success. Analysts have noticed that a daily total is roughly two percent lower than the source system reports. No error appears in any log.

What is happening, and what is the most appropriate fix?

- A The Glue job is dropping records. Increase the DPU allocation so it processes all of them
- B Unparseable values are being coerced to null. Read as a DynamicFrame and use ResolveChoice to handle both types explicitly
- C The crawler has updated the schema. Set the schema change policy to ignore changes
- D Enable job bookmarks so the job does not reprocess the affected records

◆ PAUSE AND DECIDE

What is happening, and what is the most appropriate fix?

- A

 More DPUs
Capacity for a correctness problem. The job processes every record; it misreads two percent of them.
- B

 DynamicFrame with ResolveChoice
Keeps both types alive and forces an explicit decision, instead of silently nulling values that do not parse.
- C

 Ignore schema changes
Freezes the catalog while the data keeps changing. It hides the mismatch rather than handling it.
- D

 Job bookmarks
Controls which source data is read. It has nothing to do with how a value is parsed.

THE SIGNAL

Two percent of records, two percent low, and no error. The arithmetic in the question is the diagnosis.

Converting a schema between engines

- ◆ **DMS Schema Conversion** assesses and converts schemas between database engines.
- ◆ It reports what it cannot convert, which is the genuinely useful output.
- ◆ **Stored procedures and triggers** are where manual work concentrates.
- ◆ **AWS SCT was the standalone tool.** Removed from the in-scope list in December 2025.
- ◆ **A task statement still names SCT**, so the exam guide contradicts itself. Learn the concept.

ON THE EXAM

The assessment report matters more than the conversion. It tells you how big the migration actually is.

A technical catalog answers **where is it**. A business catalog answers **should I trust it**.

Amazon SageMaker Catalog

- ◆ **Built on Amazon DataZone**, and both names appear in current material.
- ◆ **Assets** are datasets published with descriptions, owners and classifications.
- ◆ **A business glossary** defines terms once, so revenue means one thing.
- ◆ **Search across accounts** for data you did not know existed.
- ◆ **Subscription workflow**. Request access, an owner approves, permissions are granted.

ON THE EXAM

The subscription workflow is the part that matters operationally. Access becomes a request, not a ticket to an engineer.

Domains, projects and how access flows

- ◆ A **domain** is the top-level boundary, usually a business unit or the organisation.
- ◆ **Domain units** subdivide it, mirroring how teams are organised.
- ◆ **Projects** are where people and datasets come together to do work.
- ◆ **Membership of a project** is what grants access, rather than individual grants.
- ◆ Named in exam guide skills 4.1.7 and 4.5.6, both new in version 1.1.

ON THE EXAM

Access follows project membership. That is the model, and module twenty covers its security implications.

Lineage answers two questions. **Where did this number come from, and what breaks if I change this.**

How lineage is captured

- ◆ Automatically from AWS Glue, Amazon Redshift and EMR Spark executions.
- ◆ OpenLineage compatible, so other systems can push events through an API.
- ◆ Column level, not only table level. You can trace a single field.
- ◆ Versioned, so you can see the lineage as it stood at a past date.
- ◆ It follows the services that report it. A hand-written script outside them is a gap.

ON THE EXAM

Column-level lineage is what makes an impact assessment precise. Table level tells you a table is affected, not which field.

A data engineering team is about to change the data type of a column in a widely used curated table. They do not know which downstream jobs, dashboards and reports read that column, and a previous similar change broke three reports nobody had been told about. The estate uses AWS Glue for transformation, Amazon Redshift for the warehouse and Amazon Athena for ad hoc queries. The team wants to assess the impact before making the change.

What should the team use?

- ☐ A Query the AWS Glue Data Catalog for every table containing a column of that name
- ☐ B Use data lineage in Amazon SageMaker Catalog to trace downstream consumers at column level
- ☐ C Search AWS CloudTrail for API calls that referenced the table
- ☐ D Enable Glue job bookmarks so affected jobs can be reprocessed after the change

◆ PAUSE AND DECIDE

What should the team use?

- A

 Search the catalog by column name
Finds tables with a similarly named column. It says nothing about which jobs read this one, or what they feed.
- C

 CloudTrail search
Records API calls, so it shows access. It cannot show that a job read this column and derived another one from it.
- B

 Column-level lineage in SageMaker Catalog
Traces downstream from that specific column through Glue, Redshift and Athena. Impact assessment is what lineage exists for.
- D

 Job bookmarks
Concerns reprocessing after a change. The requirement is to know the impact before making it.

THE SIGNAL

"Which downstream things read this column" is the definition of downstream lineage. Nothing else answers it.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Files exist but queries return nothing	The partition is not registered in the catalog
Delay before a query scans anything	Too many partitions. Index, project, or coarsen them
A total is quietly low	Type coercion turning unparseable values into null
Every column read as a string	A crawler without a suitable classifier
Columns shifted by one	A column inserted mid-file in a positional format like CSV
Crawler cost out of proportion	Ten minute billing minimum on a frequent schedule

ON THE EXAM

The first and third rows are both silent. Nothing fails, and the answers are completely different.

Keeping a catalog useful over time

- ◆ **Catalogs rot.** Abandoned tables accumulate and nobody knows which are live.
- ◆ **Deprecate rather than delete** when a crawler finds data gone, so history survives.
- ◆ **Ownership on every asset**, so there is somebody to ask.
- ◆ **Column statistics** need refreshing as data changes, like Redshift's ANALYZE.
- ◆ **Lineage shows what is genuinely used**, which is how you find the dead tables.

ON THE EXAM

The last point is the practical one. Lineage with no downstream consumers is a table nobody reads.

Reading a catalog question quickly

- ◆ **Technical or business catalog?** Query planning, or people finding data.
- ◆ **Is a crawler necessary?** If the job knows the schema, registering at write time is cheaper.
- ◆ **Missing data or wrong data?** Partitions cause the first, type handling the second.
- ◆ **Before or after the change?** Impact assessment is lineage. Recovery is something else.
- ◆ **Athena only, or every engine?** Partition projection helps only one of those.

ON THE EXAM

Domain 2 complete. Next: Domain 3 begins with querying and analysing data.